

UNIVERSIDAD CETEC DE MÉXICO

Programación Orientada a Objetos

Actividad 3
Aplicación integradora de POO

Docente
Ing. Jessica de Aguilar

Alumno
Jesús Alfredo Rosales Orellana

Fecha de entrega
13 de octubre de 2026

Parte 1. Constructores y comportamiento

Ejercicio 1. Constructores robustos

Retomé mi proyecto de venta de libros y añadí audiolibros. En Libro validé el título, el código y el precio antes de completar la creación. No permití cadenas vacías, precios negativos ni valores booleanos como precios. Utilicé centavos enteros para expresar los importes. Inicialicé el estado vendido, la reserva, la estrategia de entrega y un bloqueo para proteger las compras de una misma instancia.

Dejé vendido=False y reservado=False como valores predeterminados. Rechacé un estado que indicara vendido y reservado al mismo tiempo. En LibroElectronico y LibroAudio utilicé super() para mantener la inicialización de los datos comunes. En ArchivoLibros inicialicé la ruta y el registro de clases. Documenté el comportamiento de las clases con cadenas de documentación, como un manual técnico breve.

Ejercicio 2. Sobrecarga y sobrescritura

Conservé calcular_total(cantidad=1) de la Actividad 2. Puedo llamar al método sin pasar una cantidad o indicando una. Entiendo que esto simula formas distintas de llamada, pero no crea varios métodos con el mismo nombre. Para la sobrescritura añadí LibroAudio, que hereda de LibroElectronico y redefine describir(). También redefiní ese método en LibroElectronico. Dejé ambos ejemplos comentados dentro de modelos.py.

Me pregunté si cambiar solamente el texto de la descripción era suficiente. Considero que sí demuestra sobrescritura, porque la misma operación heredada tiene una implementación propia. La entrega diferente quedó separada mediante composición, para no duplicar la lógica de compra en cada subclase.

Interfaces, ambiente y excepciones

Ejercicio 3. Contrato y preparación del ambiente

Definí Entrega con ABC y el método abstracto entregar(titulo). EntregaFisica y EntregaDigital cumplen ese contrato. La primera prepara un envío; la segunda habilita una descarga. No puedo instanciar Entrega directamente mientras tenga un método abstracto sin implementar (Python Software Foundation, s. f.-a).

Si entregara el proyecto a otra persona, indicaría que lo probé con Python 3.12.14. Le pediría guardar modelos.py, archivos.py y app.py en la misma carpeta, crear un entorno virtual y ejecutar python app.py desde esa carpeta. No necesitaría instalar paquetes externos porque utilicé abc, json, threading, queue, pathlib y tempfile de la biblioteca estándar. También entregaría test_app.py para ejecutar las pruebas. En Windows activaría el entorno con .venv\Scripts\Activate.ps1; en macOS o Linux usaría source .venv/bin/activate.

```
python -m venv .venv
python app.py
python -m unittest test_app -v
```

Ejercicio 4. Excepción de mi dominio

Creé StockInsuficienteError, que hereda de Exception. La lancé cuando intenté comprar una unidad física ya vendida o reservar una unidad que no estuviera disponible. En app.py capturé el error con try/except y mostré: Compra rechazada: La unidad física está agotada. Así distinguí una operación no permitida de una compra válida. Según la documentación, puedo definir excepciones propias y capturarlas por su tipo (Python Software Foundation, s. f.-b).

Parte 2. Concurrencia y entrada/salida

Ejercicio 5. Ciclo de vida de un hilo

Entiendo un hilo como una secuencia de ejecución dentro de un proceso.
Antes de start(), lo tengo creado, pero todavía no se ejecuta.
Después de start(), puede ejecutar su tarea o esperar recursos.
Con join(), hago esperar al hilo principal hasta su finalización.
Al terminar, is_alive() devuelve False y no puedo reiniciarlo.
En CPython 3.12, usar hilos no garantiza paralelismo de código Python.

Implementé tres hilos para consultar las descripciones. Utilicé Queue para reunir los resultados y join() para esperar antes de mostrarlos. No modifiqué los libros durante esas consultas. Ordené las respuestas por su índice para facilitar la lectura, sin afirmar que los hilos terminaron en ese orden. Me apoyé en la documentación de threading (Python Software Foundation, s. f.-d).

Ejercicio 6. Guardado y reconstrucción

En ArchivoLibros guardé to_dict() como JSON con codificación UTF-8. Después leí el archivo y seleccioné la clase mediante un registro de tipos. Con desde_diccionario() reconstruí el título, el código, el precio y el estado. Comprobé que una unidad vendida sigue vendida después de recuperarla. Utilicé with para cerrar los archivos y un archivo temporal para completar la escritura antes de reemplazar el destino (Python Software Foundation, s. f.-c).

Manejé archivo ausente, JSON inválido, estructura incorrecta y errores de lectura o escritura con excepciones propias. En la demostración utilicé una carpeta temporal y provoqué dos errores de lectura. Para conservar los datos, usaría una ruta fuera de TemporaryDirectory. No ejecuté escrituras concurrentes al mismo archivo.

Herencia y rediseño con composición

Ejercicio 7. Problema identificado

En la Actividad 2, Libro implementaba una compra física y LibroElectronico necesitaba sobrescribirla para evitar agotar un producto digital. Al añadir LibroAudio, advertí que seguir creando variantes podía multiplicar las excepciones a la regla de la clase base. Reproduje el problema en un ejemplo separado con tres niveles: LibroFisicoAnterior, LibroDigitalAnterior y AudioAnterior. Al no redefinir comprar(), el audiolibro recibe instrucciones para preparar una caja y un envío físico.

El siguiente fragmento es deliberadamente incorrecto y lo separé de la aplicación final como problema_herencia.py.

```
"""Reproduzco el defecto de un diseño anterior; no es mi solución final."""


class LibroFisicoAnterior:
    def comprar(self):
        return "Preparar una caja y un envío físico."


class LibroDigitalAnterior(LibroFisicoAnterior):
    # Heredo comprar() aunque necesito una descarga, no una caja.
    pass


class AudioAnterior(LibroDigitalAnterior):
    pass


if __name__ == "__main__":
    print("Problema: el audiolibro heredó una entrega física.")
    print(AudioAnterior().comprar())
```

En la solución final, cada Libro contiene una estrategia Entrega. Comprar delega la entrega y consulta su regla de inventario. Puedo inyectar otra estrategia sin reescribir comprar(). Mantuve la herencia para compartir datos y redefinir describir(), pero separé la entrega mediante composición.

Aplicación integradora y principios SOLID

Ejercicio 8. Polimorfismo

En app.py creé una lista con Libro, LibroElectronico y LibroAudio. Llamé describir() y comprar() sobre cada objeto. Las dos subclases sobrescriben describir(); la compra utiliza las estrategias de entrega. Conservé atributos encapsulados, igualdad por código, creación desde diccionarios y reserva de unidades físicas. Simulé la operación sin pagos ni envíos reales.

Responsabilidad única (SRP)

Separé los datos y las compras en modelos.py, la persistencia en archivos.py y la demostración en app.py. Sin esa separación, un cambio en JSON me obligaría a revisar la lógica de venta dentro de una misma clase.

Abierto/cerrado (OCP)

Dejé comprar() abierto a nuevas estrategias de Entrega sin modificar su algoritmo. Probé una entrega por retiro en tienda. Sin este criterio, tendría condiciones para cada tipo dentro de comprar(). Para guardar una estrategia nueva necesitaría ampliar el JSON.

Inversión de dependencias (DIP)

Permití recibir la entrega en el constructor mediante el contrato Entrega. Aunque proporcioné valores predeterminados, la compra utiliza su contrato. Sin esta decisión, crearía una entrega física dentro de comprar(). Relacioné estos criterios con la explicación de SOLID de Martin (2020).

Ejecuté seis pruebas de compras, reconstrucción, errores y estrategias. El bloqueo protege una instancia; no coordina otros procesos ni archivos compartidos.

Código: modelos.py (1 de 3)

Líneas 1 a 48. Continúo el mismo archivo en orden.

```
"""Defino libros y formas de entrega para mi aplicación de consola."""
from abc import ABC, abstractmethod
from threading import Lock


class StockInsuficienteError(Exception):
    """Comunico que una unidad física ya fue vendida."""


class Entrega(ABC):
    """Exijo una entrega y una regla de inventario a cada estrategia."""
    limitada = False

    @abstractmethod
    def entregar(self, titulo):
        pass


class EntregaFisica(Entrega):
    """Preparo un envío y marco que el inventario tiene límite."""
    limitada = True

    def entregar(self, titulo):
        return f"Preparar envío de {titulo}."


class EntregaDigital(Entrega):
    """Habilito una descarga sin agotar un inventario de unidades."""
    def entregar(self, titulo):
        return f"Habilitar descarga de {titulo}."


class Libro:
    """Inicializo una unidad, su estado y una estrategia de entrega."""
    tipo = "fisico"

    def __init__(self, titulo, codigo, precio_centavos,
                 entrega=None, vendido=False, reservado=False):
        for nombre, valor in (("título", titulo), ("código", codigo)):
            if not isinstance(valor, str) or not valor.strip():
                raise ValueError(f"El {nombre} debe ser un texto no vacío.")
        if type(precio_centavos) is not int or precio_centavos < 0:
            raise ValueError("El precio debe ser un entero no negativo.")
        if type(vendido) is not bool:
            raise ValueError("El estado vendido debe ser booleano.")
        if type(reservado) is not bool or (vendido and reservado):
            raise ValueError("La reserva debe ser válida y no estar vendida.")
        self.titulo = titulo.strip()
```

Código: modelos.py (2 de 3)

Líneas 49 a 96. Continúo el mismo archivo en orden.

```
        self.__codigo = codigo.strip()
        self.__precio = precio_centavos
        # COMPOSICIÓN y DIP: recibo una estrategia por su contrato.
        self._entrega = entrega if entrega is not None else EntregaFisica()
        if not isinstance(self._entrega, Entrega):
            raise TypeError("La entrega debe cumplir el contrato Entrega.")
        self.__vendido = vendido
        self.__reservado = reservado
        self.__lock = Lock()

    @property
    def codigo(self):
        return self.__codigo

    @property
    def vendido(self):
        with self.__lock:
            return self.__vendido

    @classmethod
    def desde_diccionario(cls, datos):
        return cls(datos["titulo"], datos["codigo"], datos["precio_centavos"],
                   vendido=datos["vendido"], reservado=datos["reservado"])

    def __eq__(self, otro):
        if not isinstance(otro, Libro):
            return NotImplemented
        return self.codigo == otro.codigo

    def reservar(self):
        with self.__lock:
            if not self._entrega.limitada:
                return "El producto digital no necesita reserva."
            if self.__vendido or self.__reservado:
                raise StockInsuficienteError("No puedo reservar esta unidad.")
            self.__reservado = True
            return f"Reservé {self.titulo}."

    def describir(self):
        return f"Físico: {self.titulo} [{self.codigo}]"

    def comprar(self):
        # Evito dos ventas simultáneas de la misma unidad física.
        with self.__lock:
            if self._entrega.limitada and self.__vendido:
                raise StockInsuficienteError("La unidad física está agotada.")
            mensaje = self._entrega.entregar(self.titulo)
            if self._entrega.limitada:
```

Código: modelos.py (3 de 3)

Líneas 97 a 142. Continúo el mismo archivo en orden.

```
                self.__vendido = True
                self.__reservado = False
            return mensaje

    def calcular_total(self, cantidad=1):
        # SOBRECARGA SIMULADA (Actividad 2): una sola implementación
        # admite llamadas con o sin cantidad gracias al valor por defecto.
        if type(cantidad) is not int or cantidad <= 0:
            raise ValueError("La cantidad debe ser un entero positivo.")
        return self.__precio * cantidad

    def to_dict(self):
        with self.__lock:
            return {"tipo": self.tipo, "titulo": self.titulo,
                    "codigo": self.codigo, "precio_centavos": self.__precio,
                    "vendido": self.__vendido,
                    "reservado": self.__reservado}


class LibroElectronico(Libro):
    """Inicializo un producto digital con entrega descargable."""
    tipo = "electronico"

    def __init__(self, titulo, codigo, precio_centavos,
                 entrega=None, vendido=False, reservado=False):
        super().__init__(titulo, codigo, precio_centavos,
                         entrega if entrega is not None else EntregaDigital(),
                         vendido, reservado)

    def describir(self):
        # SOBRESCRITURA: reemplazo la descripción heredada.
        return f"Electrónico: {self.titulo} [{self.codigo}]"


class LibroAudio(LibroElectronico):
    """Reutilizo la entrega digital y agrego una descripción de audio."""
    tipo = "audio"

    def __init__(self, titulo, codigo, precio_centavos,
                 entrega=None, vendido=False, reservado=False):
        super().__init__(titulo, codigo, precio_centavos,
                         entrega, vendido, reservado)

    def describir(self):
        # SOBRESCRITURA ADICIONAL: mismo nombre, diferente comportamiento.
        return f"Audiolibro: {self.titulo} [{self.codigo}]"
```

Código: archivos.py (1 de 2)

Líneas 1 a 48. Continúo el mismo archivo en orden.

```
"""Separo la entrada/salida de las reglas de compra (SRP)."""
import json
from pathlib import Path
from modelos import Libro, LibroElectronico, LibroAudio


class ArchivoNoEncontradoError(Exception):
    """Comunico que no existe el archivo solicitado."""


class DatosCorruptosError(Exception):
    """Comunico JSON inválido o datos que incumplen el modelo."""


class ArchivoInaccesibleError(Exception):
    """Comunico errores del sistema al leer o escribir."""


class ArchivoLibros:
    """Inicializo una ruta y el registro de tipos reconstruibles."""
    def __init__(self, ruta):
        self.ruta = Path(ruta)
        self.tipos = {"fisico": Libro, "electronico": LibroElectronico,
                      "audio": LibroAudio}

    def guardar(self, libro):
        temporal = self.ruta.with_suffix(self.ruta.suffix + ".tmp")
        try:
            with temporal.open("w", encoding="utf-8") as archivo:
                json.dump(libro.to_dict(), archivo, ensure_ascii=False,
                          indent=2)
            # Reemplazo el destino cuando terminé la escritura.
            temporal.replace(self.ruta)
        except OSError as error:
            raise ArchivoInaccesibleError("No pude guardar el libro.") from error
        finally:
            # Limpieza sin ocultar la causa principal de un fallo.
            try:
                temporal.unlink(missing_ok=True)
            except OSError:
                pass

    def leer(self):
        try:
            with self.ruta.open(encoding="utf-8") as archivo:
                datos = json.load(archivo)
        except FileNotFoundError as error:
            raise ArchivoNoEncontradoError("No encontré el archivo.") from error
```

Código: archivos.py (2 de 2)

Líneas 49 a 59. Continúo el mismo archivo en orden.

```
        except (json.JSONDecodeError, UnicodeError) as error:
            raise DatosCorruptosError(
                "El archivo no contiene JSON válido.") from error
        except OSError as error:
            raise ArchivoInaccesibleError("No pude leer el archivo.") from error
        try:
            clase = self.tipos[datos["tipo"]]
            return clase.desde_diccionario(datos)
        except (KeyError, TypeError, ValueError) as error:
            raise DatosCorruptosError(
                "Los datos del libro son incorrectos.") from error
```

Código: app.py (1 de 2)

Líneas 1 a 48. Continúo el mismo archivo en orden.

```
"""Ejecuto compras, persistencia y consultas concurrentes de demostración."""
from pathlib import Path
from queue import Queue
from threading import Thread
from tempfile import TemporaryDirectory
from modelos import Libro, LibroElectronico, LibroAudio, StockInsuficienteError
from archivos import (ArchivoLibros, ArchivoNoEncontradoError,
                      DatosCorruptosError, ArchivoInaccesibleError)


def describir_en_hilos(libros):
    resultados = Queue()

    def tarea(indice, libro):
        try:
            resultados.put((indice, libro.describir()))
        except Exception as error:
            resultados.put((indice, f"Error en consulta: {error}"))

    hilos = [Thread(target=tarea, args=(i, libro), name=f"consulta-{i}")
             for i, libro in enumerate(libros)]
    print("Hilos creados: todavía no iniciados.")
    for hilo in hilos:
        hilo.start()
    for hilo in hilos:
        hilo.join()
    print(f"Hilos terminados: {all(not h.is_alive() for h in hilos)}")
    return [texto for _, texto in sorted(
        resultados.get() for _ in libros)]


def main():
    libros = [Libro("Python desde cero", "FIS-003", 2500),
              LibroElectronico("Bases de datos", "ELE-003", 1200),
              LibroAudio("Aprender escuchando", "AUD-001", 1800)]
    print("1. POLIMORFISMO Y COMPOSICIÓN")
    print(libros[0].reservar())
    for libro in libros:
        print(libro.describir())
        print(libro.comprar())
    print("2. EXCEPCIÓN DE DOMINIO")
    try:
        libros[0].comprar()
    except StockInsuficienteError as error:
        print(f"Compra rechazada: {error}")
    print("3. ARGUMENTOS POR DEFECTO")
    print(f"Una copia digital: {libros[1].calcular_total()} centavos")
    print(f"Tres copias digitales: {libros[1].calcular_total(3)} centavos")
```

Código: app.py (2 de 2)

Líneas 49 a 77. Continúo el mismo archivo en orden.

```
    print("4. CONSULTAS CONCURRENTES")
    for resultado in describir_en_hilos(libros):
        print(resultado)
    print("5. GUARDADO Y RECONSTRUCCIÓN")
    with TemporaryDirectory() as carpeta:
        ruta = Path(carpeta) / "libro.json"
        repositorio = ArchivoLibros(ruta)
        try:
            repositorio.guardar(libros[0])
            print(ruta.read_text(encoding="utf-8"))
            recuperado = repositorio.leer()
            print(f"Recuperé: {recuperado.describir()}")
            print(f"Conservé vendido: {recuperado.vendido}")
            ruta.write_text("{JSON incompleto", encoding="utf-8")
            try:
                repositorio.leer()
            except DatosCorruptosError as error:
                print(f"Lectura rechazada: {error}")
            ruta.unlink()
            try:
                repositorio.leer()
            except ArchivoNoEncontradoError as error:
                print(f"Lectura rechazada: {error}")
        except ArchivoInaccesibleError as error:
            print(f"Error de almacenamiento: {error}")


if __name__ == "__main__":
    main()
```

Código: test_app.py (1 de 2)

Líneas 1 a 48. Continúo el mismo archivo en orden.

```
"""Compruebo invariantes de compras, reconstrucción y concurrencia."""
import unittest
from pathlib import Path
from tempfile import TemporaryDirectory
from threading import Thread
from queue import Queue
from modelos import Libro, LibroAudio, StockInsuficienteError, Entrega
from archivos import (ArchivoLibros, DatosCorruptosError,
                      ArchivoNoEncontradoError, ArchivoInaccesibleError)


class PruebasAplicacion(unittest.TestCase):
    def test_venta_concurrente_unica(self):
        libro, resultados = Libro("Python", "F-1", 100), Queue()

        def comprar():
            try:
                libro.comprar()
                resultados.put("venta")
            except StockInsuficienteError:
                resultados.put("rechazo")

        hilos = [Thread(target=comprar) for _ in range(2)]
        for hilo in hilos:
            hilo.start()
        for hilo in hilos:
            hilo.join()
        self.assertCountEqual([resultados.get(), resultados.get()],
                              ["venta", "rechazo"])

    def test_recupero_subclase_y_estado(self):
        with TemporaryDirectory() as carpeta:
            repositorio = ArchivoLibros(Path(carpeta) / "libro.json")
            audio = LibroAudio("Audio", "A-1", 300)
            repositorio.guardar(audio)
            recuperado = repositorio.leer()
            self.assertIsInstance(recuperado, LibroAudio)
            self.assertEqual(recuperado.to_dict(), audio.to_dict())
            fisico = Libro("Python", "F-1", 100)
            fisico.comprar()
            repositorio.guardar(fisico)
            with self.assertRaises(StockInsuficienteError):
                repositorio.leer().comprar()

    def test_json_invalido_y_archivo_ausente(self):
        with TemporaryDirectory() as carpeta:
            ruta = Path(carpeta) / "libro.json"
            repositorio = ArchivoLibros(ruta)
```

Código: test_app.py (2 de 2)

Líneas 49 a 87. Continúo el mismo archivo en orden.

```
            with self.assertRaises(ArchivoNoEncontradoError):
                repositorio.leer()
            for texto in ("{incompleto", "[]", '{"tipo":"desconocido"}'):
                ruta.write_text(texto, encoding="utf-8")
                with self.assertRaises(DatosCorruptosError):
                    repositorio.leer()

    def test_interfaz_y_argumentos(self):
        with self.assertRaises(TypeError):
            Entrega()
        with self.assertRaises(ValueError):
            Libro("Python", "F-1", -1)
        audio = LibroAudio("Audio", "A-1", 300)
        self.assertEqual(audio.calcular_total(), 300)
        self.assertEqual(audio.calcular_total(2), 600)
        self.assertEqual(audio.comprar(), audio.comprar())

    def test_reserva_y_estrategia_intercambiable(self):
        class Retiro(Entrega):
            limitada = True

            def entregar(self, titulo):
                return f"Retirar {titulo} en tienda."

        libro = Libro("Python", "F-1", 100, entrega=Retiro())
        libro.reservar()
        self.assertTrue(libro.to_dict()["reservado"])
        self.assertEqual(libro.comprar(), "Retirar Python en tienda.")
        self.assertFalse(libro.to_dict()["reservado"])

    def test_comunico_fallo_de_escritura(self):
        with TemporaryDirectory() as carpeta:
            ruta = Path(carpeta) / "carpeta_ausente" / "libro.json"
            with self.assertRaises(ArchivoInaccesibleError):
                ArchivoLibros(ruta).guardar(Libro("Python", "F-1", 100))


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Evidencias de ejecución

Figura 1

Compras, polimorfismo y consultas con hilos

```
$ python app.py
1. POLIMORFISMO Y COMPOSICIÓN
Reservé Python desde cero.
Físico: Python desde cero [FIS-003]
Preparar envío de Python desde cero.
Electrónico: Bases de datos [ELE-003]
Habilitar descarga de Bases de datos.
Audiolibro: Aprender escuchando [AUD-001]
Habilitar descarga de Aprender escuchando.
2. EXCEPCIÓN DE DOMINIO
Compra rechazada: La unidad física está agotada.
3. ARGUMENTOS POR DEFECTO
Una copia digital: 1200 centavos
Tres copias digitales: 3600 centavos
4. CONSULTAS CONCURRENTES
Hilos creados: todavía no iniciados.
Hilos terminados: True
Físico: Python desde cero [FIS-003]
Electrónico: Bases de datos [ELE-003]
Audiolibro: Aprender escuchando [AUD-001]
```

Nota. Exporté a imagen la salida real de consola de esta ejecución; no corresponde a una ventana gráfica. Mostré la reserva, las tres descripciones, las entregas, la excepción de inventario y el cierre de los hilos.

Evidencias de ejecución

Figura 2

JSON, reconstrucción y manejo de errores

```
5. GUARDADO Y RECONSTRUCCIÓN
{
  "tipo": "fisico",
  "titulo": "Python desde cero",
  "codigo": "FIS-003",
  "precio_centavos": 2500,
  "vendido": true,
  "reservado": false
}
Recuperé: Físico: Python desde cero [FIS-003]
Conservé vendido: True
Lectura rechazada: El archivo no contiene JSON válido.
Lectura rechazada: No encontré el archivo.
```

Nota. Exporté a imagen la salida real de consola de esta ejecución; no corresponde a una ventana gráfica. Comprobé que el estado vendido se conserva y comuniqué los errores de JSON inválido y archivo ausente.

Evidencias de ejecución

Figura 3

Pruebas de la aplicación

```
$ python -m unittest test_app -v
test_comunico_fallo_de_escritura
(test_app.PruebasAplicacion.test_comunico_fallo_de_escritura) ... ok
test_interfaz_y_argumentos (test_app.PruebasAplicacion.test_interfaz_y_argumentos)
... ok
test_json_invalido_y_archivo_ausente
(test_app.PruebasAplicacion.test_json_invalido_y_archivo_ausente) ... ok
test_recupero_subclase_y_estado
(test_app.PruebasAplicacion.test_recupero_subclase_y_estado) ... ok
test_reserva_y_estrategia_intercambiable
(test_app.PruebasAplicacion.test_reserva_y_estrategia_intercambiable) ... ok
test_venta_concurrente_unica
(test_app.PruebasAplicacion.test_venta_concurrente_unica) ... ok

----------------------------------------------------------------------
Ran 6 tests in 0.003s

OK
```

Nota. Exporté a imagen la salida real de consola de esta ejecución; no corresponde a una ventana gráfica. Obtuve seis pruebas aprobadas. También comprobé una venta concurrente de una unidad, una estrategia nueva y un error de escritura.

Conclusión: evolución de mi proyecto

En la Actividad 1 empecé con una clase Libro para representar un producto físico o electrónico. Definí atributos, reservas y compras. Comprendí que dos objetos pueden tener el mismo código y ser iguales en valor, aunque sean instancias distintas. También practiqué la creación desde diccionarios y el encapsulamiento. En ese momento me concentré principalmente en que los objetos tuvieran datos y comportamientos relacionados.

En la Actividad 2 revisé la calidad del código. Me resultó importante evitar estados contradictorios y rechazar datos incorrectos. Incorporé herencia, polimorfismo y una clase abstracta, y agrupé libros en un catálogo. Además, publiqué el proyecto, las pruebas y las instrucciones en GitHub. Con esa etapa comprendí que escribir una clase no basta: también necesito poder explicar, comprobar y conservar los cambios que realizo.

En esta entrega añadí audiolibros, entrada/salida y consultas con hilos. Al reconstruir un libro desde JSON, comprobé que conservar el estado es tan importante como recuperar el título. También descubrí un límite de mi herencia anterior: la entrega física no debía determinar la compra de todos los formatos. Por eso separé esa responsabilidad mediante composición y mantuve una interfaz común. Me pregunté si añadir más clases complicaría el programa, pero consideré que en este caso cada separación resolvió un problema concreto.

Cerré el proyecto con una aplicación de consola y seis pruebas aprobadas. No considero que esté lista para una tienda real: todavía necesitaría usuarios, pagos y un almacenamiento más completo. Mi avance principal fue pasar de una clase aislada a una solución que puedo ejecutar, revisar y extender con responsabilidades más claras.

Referencias

Martin, R. C. (2020, 18 de octubre). *Solid relevance*. The Clean Code Blog. https://blog.cleancoder.com/uncle-bob/2020/10/18/Solid-Relevance.html

Python Software Foundation. (s. f.-a). *abc - Abstract base classes*. Python documentation. https://docs.python.org/3.12/library/abc.html

Python Software Foundation. (s. f.-b). *Errors and exceptions*. Python documentation. https://docs.python.org/3.12/tutorial/errors.html

Python Software Foundation. (s. f.-c). *json - JSON encoder and decoder*. Python documentation. https://docs.python.org/3.12/library/json.html

Python Software Foundation. (s. f.-d). *threading - Thread-based parallelism*. Python documentation. https://docs.python.org/3.12/library/threading.html

Rosales Orellana, J. A. [JAROrellana-SV]. (2026). *POO-clase-de-CETEC* [Repositorio de código]. GitHub. https://github.com/JAROrellana-SV/POO-clase-de-CETEC

Mi repositorio contiene el trabajo previo de la Actividad 2. Esta entrega incluye el código final en el PDF y en el paquete de archivos; no afirmo haber publicado esta nueva versión en GitHub.